

Implementation and Defense Against Adversarial Attacks in Image Classification Model



**Presented by
Umesh Kumar
23MT0430**

**Under Supervision
Of
Prof. Rajendra Pamula**

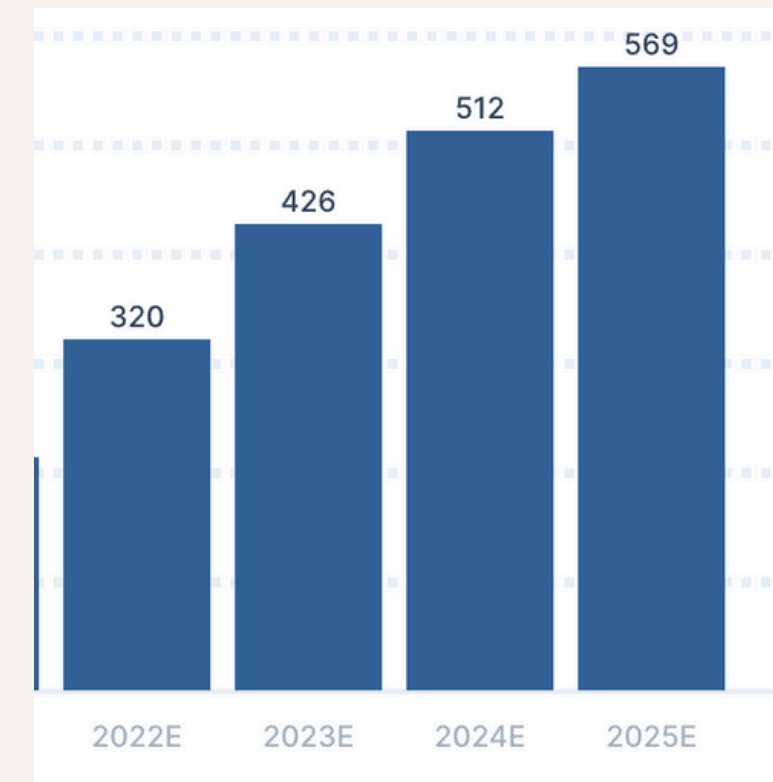
ADVERSARIAL ATTACKS



- Small, intentional perturbations to input images designed to mislead machine learning models.
- These perturbations are often imperceptible to the human eye but can significantly impact model predictions.

Impact On Real Life

- Autonomous vehicle operation an incorrect identification of traffic signs can produce severe accidents.
- Incorrect image classifications within medical scenarios could cause the misdiagnosis along with improper treatment methods.



US 2022 - 25

PROBLEM STATEMENT

- This project aims to explore the effectiveness of adversarial attacks, specifically the Fast Gradient Sign Method (FGSM), on a deep learning based image classification model
- Implimenting and Testing Defense mechanisms to enhance model robustnes
- The problem addressed is two fold:
 - 1) Evaluating Vulnerability
 - 2) Implimenting and Testing Defenses

Proposed Work

Evaluating Vulnerability :

- FGSM Attack

Implimenting and Testing Defenses :

- Adversarial Training
- Defense Layer (DefendedModel)
- Noise Injection Techniques
- Multiple Epsilons Training

Adversarial Attack - FGSM

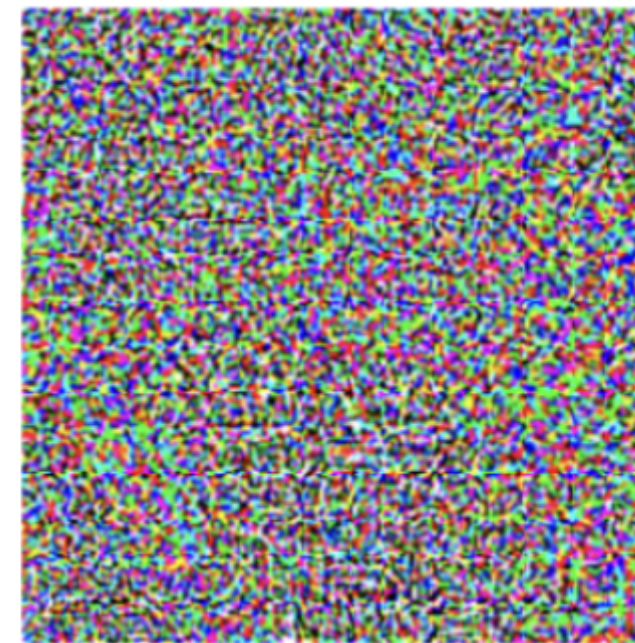
$$x_{\text{adv}} = x + \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y))$$

- Generates adversarial examples by adding noise in the direction of the gradient.
- Formula: `perturbed_image = original_image + epsilon * sign(data_grad)`
- Epsilon controls the attack strength (i.e., perturbation level).



x
“panda”

$+ .007 \times$



$\text{sign}(\nabla_x J(\theta, x, y))$
“nematode”

$=$

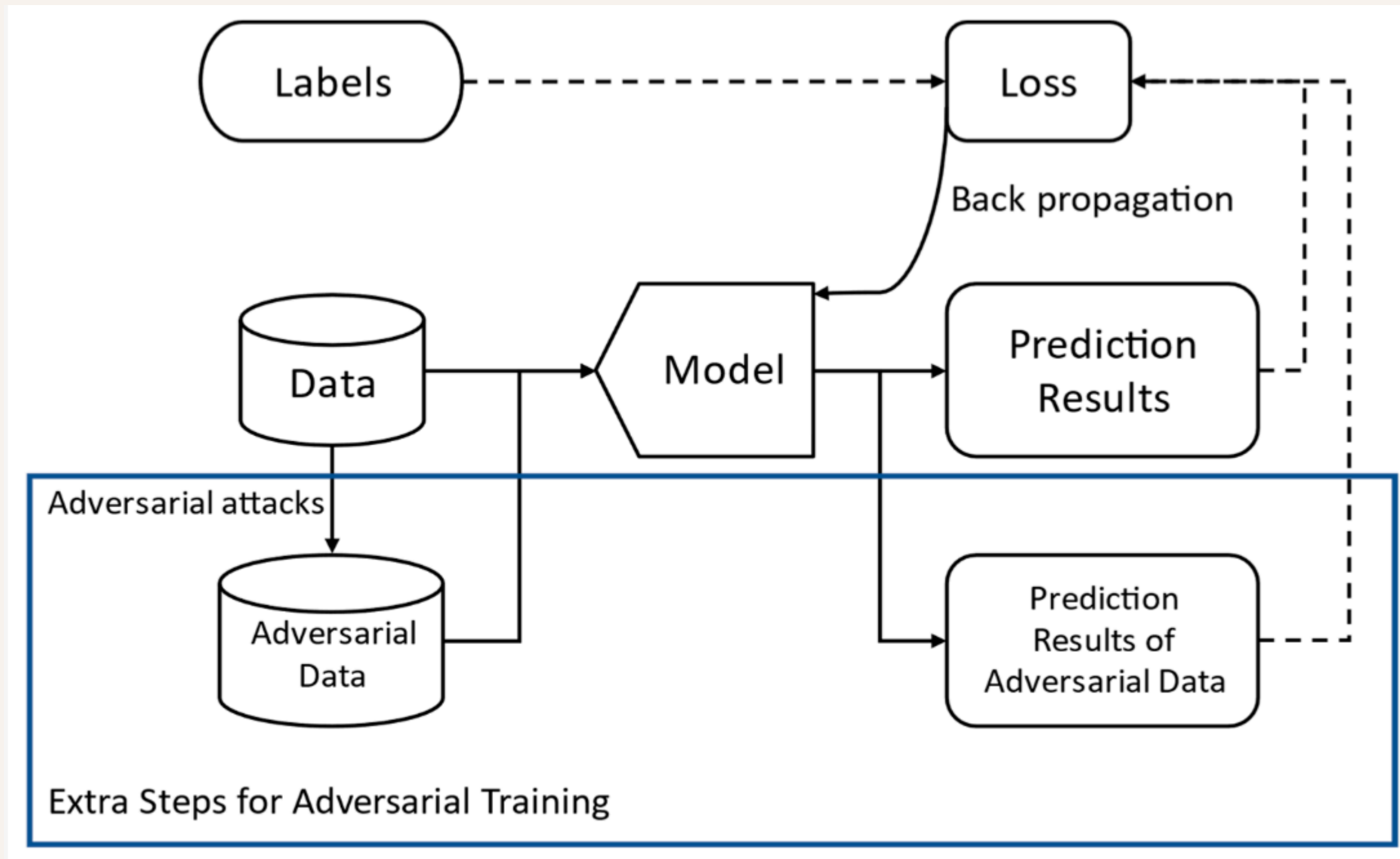


$x + \epsilon \text{sign}(\nabla_x J(\theta, x, y))$
“gibbon”

Example of FGSM attack.

Adding a small perturbation with $\epsilon = 0.007$ in panda image encoding successfully causes misclassification, while remaining imperceptible to the human eye.

Adversarial Defense Using Adversarial Training



Results After Adversarial Training

	precision	recall	f1-score	support
buildings	0.92	0.92	0.92	349
forest	0.99	0.99	0.99	379
glacier	0.88	0.88	0.88	442
mountain	0.89	0.87	0.88	420
sea	0.93	0.96	0.95	408
street	0.93	0.94	0.93	400
accuracy			0.92	2398
macro avg	0.92	0.93	0.93	2398
weighted avg	0.92	0.92	0.92	2398

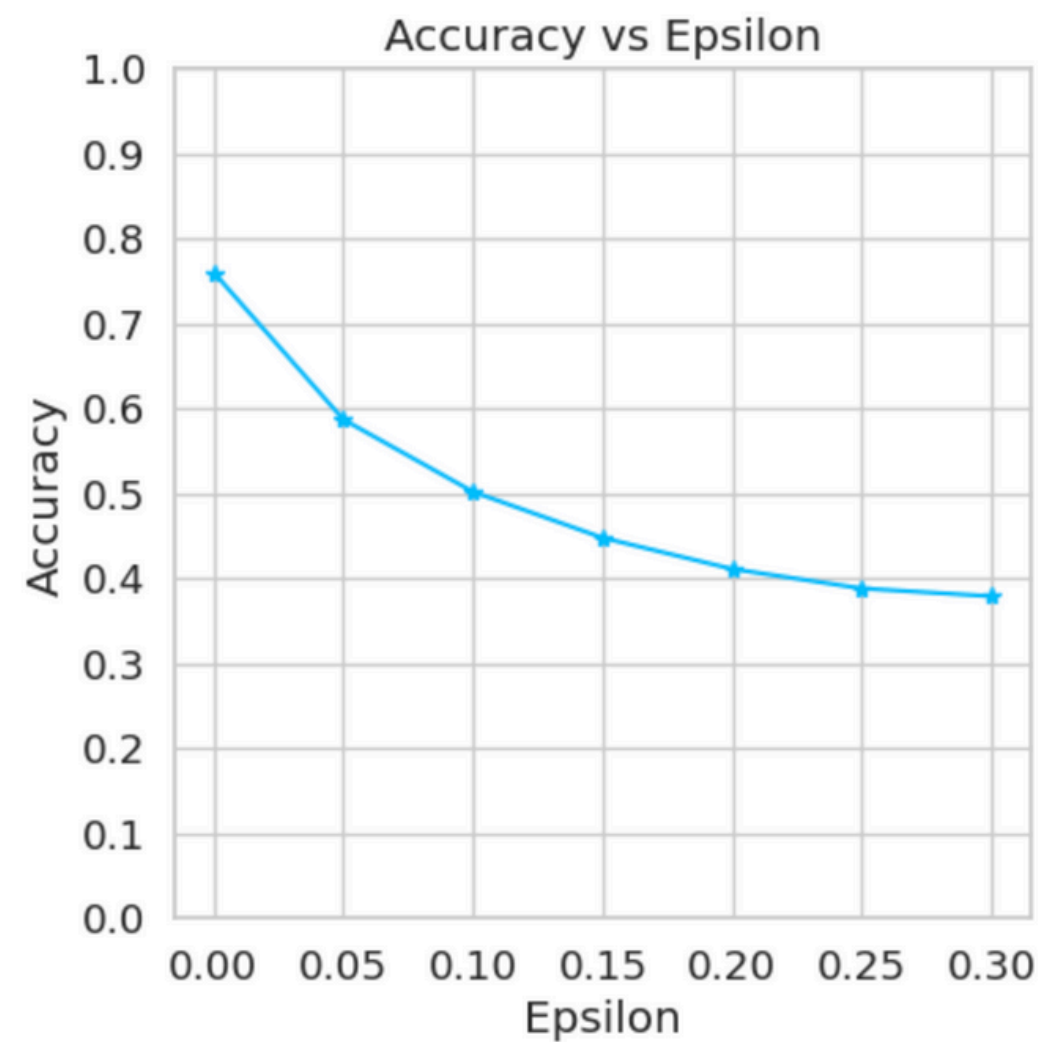
Accuraccy of ResNet-34 before Attack

```
For Epsilon =0.3
/usr/local/lib/python3.10/dist-packages/torch/utils/data/
warnings.warn(_create_warning_msg(
precision    recall  f1-score   support

 buildings    0.43    0.66    0.52     349
   forest    0.65    0.45    0.54     379
  glacier    0.28    0.32    0.30     442
 mountain    0.49    0.30    0.37     420
     sea    0.28    0.51    0.36     408
   street    0.32    0.08    0.12     400

 accuracy                0.38     2398
 macro avg              0.41    0.39    0.37     2398
 weighted avg           0.40    0.38    0.36     2398
```

Accuraccy of ResNet-34 After Attack



Accuracy vs Epsilon

	precision	recall	f1-score	support
buildings	0.76	0.84	0.80	349
forest	0.95	0.89	0.92	379
glacier	0.84	0.70	0.76	442
mountain	0.82	0.72	0.76	420
sea	0.70	0.89	0.79	408
street	0.82	0.82	0.82	400
accuracy			0.81	2398
macro avg	0.81	0.81	0.81	2398
weighted avg	0.81	0.81	0.81	2398

Accuracy After Adversarial Training

Problem With Adversarial Training

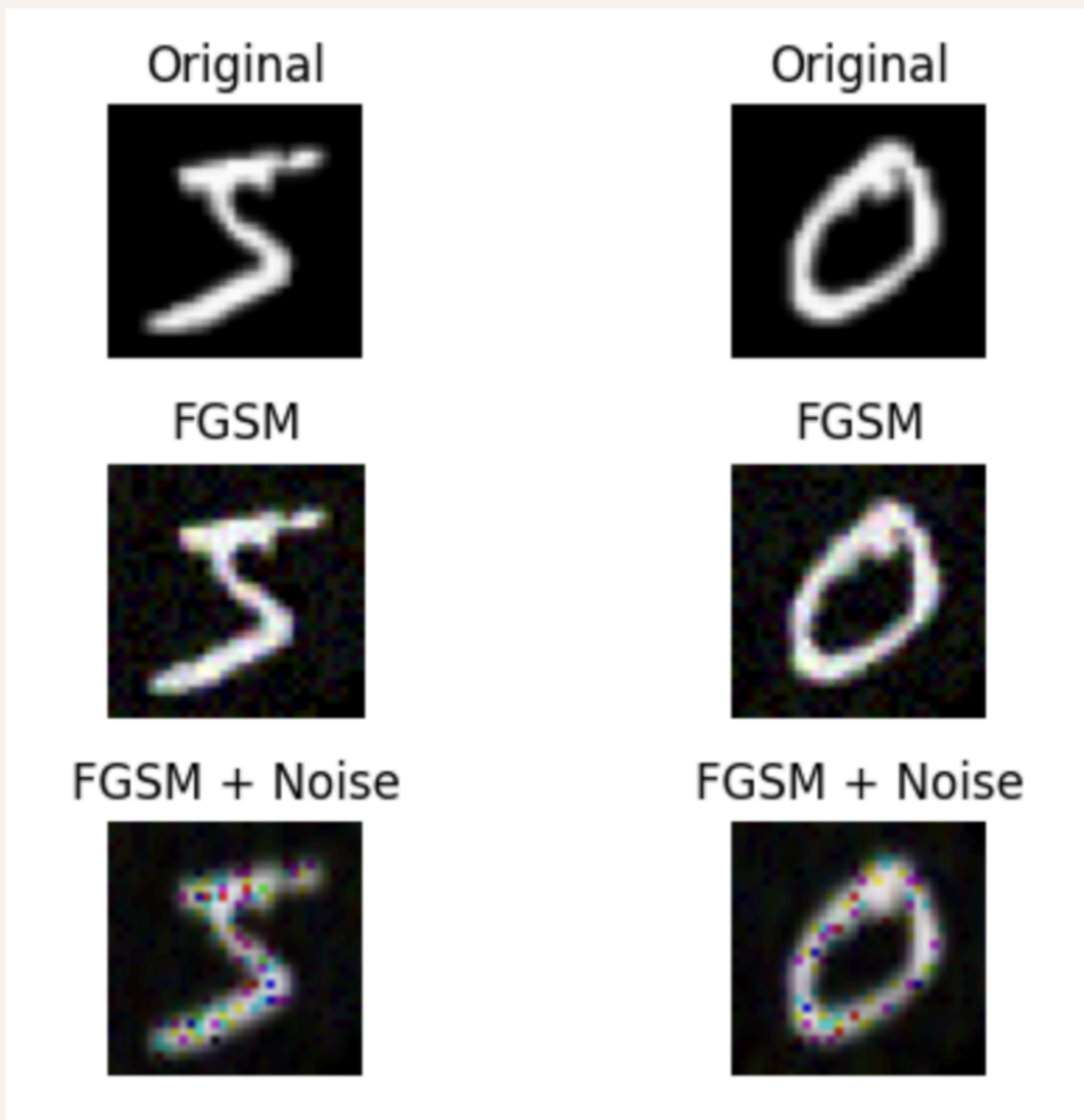
- More than one training.
- Wastage of resources(GPUs).
- Time taking
- Hard to tune(epsilon value).

Adversarial Defense Using Noise Injection

- It aim to harden the model during training
- One time training

Noise Injection Techniques

- Random Gaussian Noise (RGN): helps models to generalize better even under minor distortions
- Stochastic Motion Blur (SMB): make models less sensitive to sharp changes or small perturbations.
- Stochastic Glass Blur (SGB): remove adversarial patterns that depend on fine pixel arrangements.



Images After FGSM And Noise Injection.

Clean Test Accuracy: 92.40%
FGSM + Fixed Noise Accuracy: 74.40%

Accuracy After Noise Injection.

Defense Layers

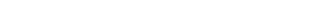
- Clean the inputs before feeding to the CNN model
- One time training
- Various techniques are :
 - Using Gaussian Blur: Smooths the image by averaging pixels with their neighbors using a Gaussian kernel.
 - Using JPEG Compression: Compresses the image using JPEG format and then decompresses it back.
 - Using Median Filtering (Input Denoising): Each pixel is replaced by the median of its neighboring pixels.

Epoch 20/20

422/422 ————— 5s 9ms/step - accuracy: 0.9982

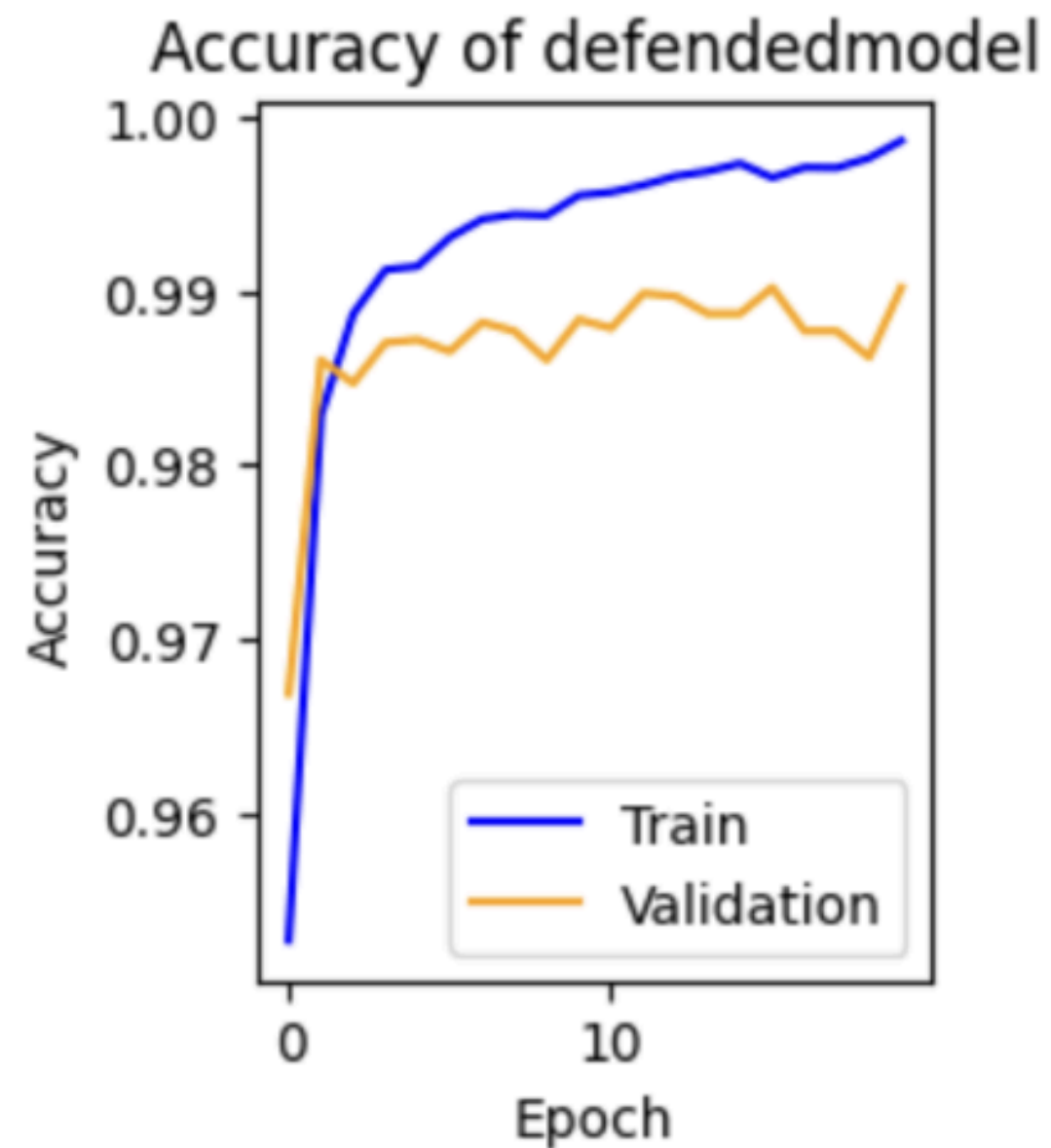
313/313 **2s 4ms/step - accuracy: 0.9911**

Accuracy on test data is : 99.19999837875366

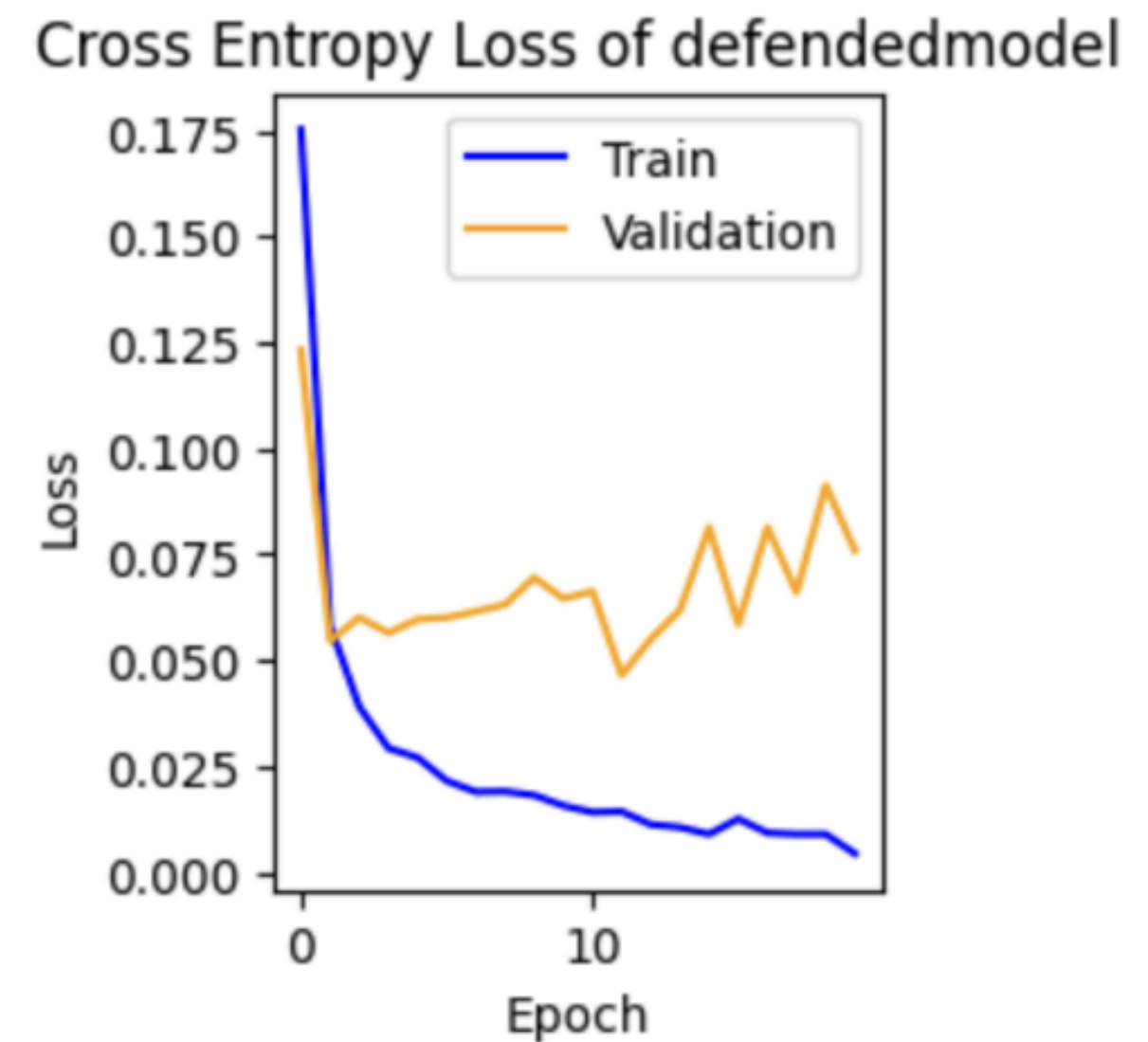
4/4  1s 215ms/step

Accuracy on perturbed images: 67.00%

DefendedModel Accuracy



Accuracy vs Epoch



Loss vs Epoch

```
4/4 _____ 0s 8ms/step
Epsilon: 0.000 => Accuracy after Gaussian blur: 98.00%
4/4 _____ 0s 8ms/step
Epsilon: 0.007 => Accuracy after Gaussian blur: 98.00%
4/4 _____ 0s 9ms/step
Epsilon: 0.010 => Accuracy after Gaussian blur: 98.00%
4/4 _____ 0s 8ms/step
Epsilon: 0.020 => Accuracy after Gaussian blur: 97.00%
4/4 _____ 0s 11ms/step
Epsilon: 0.030 => Accuracy after Gaussian blur: 97.00%
4/4 _____ 0s 8ms/step
Epsilon: 0.050 => Accuracy after Gaussian blur: 93.00%
4/4 _____ 0s 8ms/step
Epsilon: 0.100 => Accuracy after Gaussian blur: 85.00%
4/4 _____ 0s 13ms/step
Epsilon: 0.200 => Accuracy after Gaussian blur: 72.00%
4/4 _____ 0s 8ms/step
Epsilon: 0.300 => Accuracy after Gaussian blur: 52.00%
```

Using GaussianBlur Defense Layer

```
--- Median Denoising Defense ---
4/4 _____ 0s 8ms/step
Epsilon: 0.000 => Accuracy after Median Denoising: 100.00%
4/4 _____ 0s 11ms/step
Epsilon: 0.007 => Accuracy after Median Denoising: 100.00%
4/4 _____ 0s 8ms/step
Epsilon: 0.010 => Accuracy after Median Denoising: 100.00%
4/4 _____ 0s 8ms/step
Epsilon: 0.020 => Accuracy after Median Denoising: 98.00%
4/4 _____ 0s 12ms/step
Epsilon: 0.030 => Accuracy after Median Denoising: 97.00%
4/4 _____ 0s 8ms/step
Epsilon: 0.050 => Accuracy after Median Denoising: 92.00%
4/4 _____ 0s 8ms/step
Epsilon: 0.100 => Accuracy after Median Denoising: 81.00%
4/4 _____ 0s 11ms/step
Epsilon: 0.200 => Accuracy after Median Denoising: 57.00%
4/4 _____ 0s 8ms/step
Epsilon: 0.300 => Accuracy after Median Denoising: 32.00%
```

Using Median Denoising Defense Layer

```
--- JPEG Compression Defense ---
4/4 _____ 0s 8ms/step
Epsilon: 0.000 => Accuracy after JPEG compression: 99.00%
4/4 _____ 0s 9ms/step
Epsilon: 0.007 => Accuracy after JPEG compression: 99.00%
4/4 _____ 0s 8ms/step
Epsilon: 0.010 => Accuracy after JPEG compression: 99.00%
4/4 _____ 0s 9ms/step
Epsilon: 0.020 => Accuracy after JPEG compression: 99.00%
4/4 _____ 0s 8ms/step
Epsilon: 0.030 => Accuracy after JPEG compression: 98.00%
4/4 _____ 0s 8ms/step
Epsilon: 0.050 => Accuracy after JPEG compression: 94.00%
4/4 _____ 0s 8ms/step
Epsilon: 0.100 => Accuracy after JPEG compression: 79.00%
4/4 _____ 0s 8ms/step
Epsilon: 0.200 => Accuracy after JPEG compression: 63.00%
4/4 _____ 0s 8ms/step
Epsilon: 0.300 => Accuracy after JPEG compression: 31.00%
```

Using JPEGCompression Defense Layer

Training with Multiple Epsilons

- Rather than using a single, fixed perturbation strength ϵ , adversarial samples were generated using multiple epsilon values
- Values used are $\epsilon \in \{0.01, 0.05, 0.1, 0.2, 0.3\}$.
- This approach led to a substantial improvement in the model's robustness to adversarial attacks, particularly in the lower to mid-range epsilon values (e.g., 0.01–0.1).
- Compared to the baseline model, the accuracy showed less significant degradation for higher epsilon values

```

4/4 _____ 0s 8ms/step
Epsilon: 0.000 => Accuracy on perturbed images: 99.00%
4/4 _____ 0s 8ms/step
Epsilon: 0.007 => Accuracy on perturbed images: 99.00%
4/4 _____ 0s 12ms/step
Epsilon: 0.010 => Accuracy on perturbed images: 99.00%
4/4 _____ 0s 8ms/step
Epsilon: 0.020 => Accuracy on perturbed images: 98.00%
4/4 _____ 0s 9ms/step
Epsilon: 0.030 => Accuracy on perturbed images: 97.00%
4/4 _____ 0s 13ms/step
Epsilon: 0.050 => Accuracy on perturbed images: 90.00%
4/4 _____ 0s 8ms/step
Epsilon: 0.100 => Accuracy on perturbed images: 78.00%
4/4 _____ 0s 8ms/step
Epsilon: 0.200 => Accuracy on perturbed images: 59.00%
4/4 _____ 0s 8ms/step
Epsilon: 0.300 => Accuracy on perturbed images: 31.00%

```

Accuracy On
Various Epsilon

```

--- Accuracy on FGSM Adversarial Examples (Post-Adversarial Train
4/4 _____ 0s 8ms/step
Epsilon: 0.010 => Accuracy after adversarial training: 99.00%
4/4 _____ 0s 9ms/step
Epsilon: 0.050 => Accuracy after adversarial training: 90.00%
4/4 _____ 0s 8ms/step
Epsilon: 0.100 => Accuracy after adversarial training: 84.00%
4/4 _____ 0s 8ms/step
Epsilon: 0.200 => Accuracy after adversarial training: 73.00%
4/4 _____ 0s 8ms/step
Epsilon: 0.300 => Accuracy after adversarial training: 56.00%

```

Accuracy After Epsilon Training

Comparisions Among Defences

Technique	Accuracy
WITHOUT ATTACK	92%
FGSM ATTACK	38%
ADVERSEARIAL TRAINING	81%
USING GAUSSIAN BLUR (eps = 0.1)	85%
USING JPEG COMPRESSION (eps = 0.1)	81%
USING MEDIAN FILTERING (Input Denoising) (eps = 0.1)	79%
NOISE INJECTION	74%
EPSILON TRAINING	84%

Future Scope

- More advanced and stronger attacks, including PGD, DeepFool
- Design of adaptive or learnable defensive schemes that have the capability of responding to varied perturbations dynamically
- Experimenting on these defense strategies with larger and more diverse sets, e.g., CIFAR-10, CIFAR-100, or domain-specific datasets
- Deploying these methods in federated learning settings may provide solutions to practical security issues

Thank you